

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN**  
**COURSE CODE: CSA1104**

---

### COURSE OUTCOME COVERED

**CO2:** Design UML diagrams to represent system requirements and architecture.  
(BL3)

*Assessment Tool Weightage: 40%*

---

**QUESTIONS: Any 4**

**Total Marks:40**

Annexure A

### ANALYTICAL PROBLEM SOLVING

---

#### ***Code of Conduct:***

*I, Judy Allen J (192520055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

#### ***Signature of the student:***

*Judy Allen J*

## **1. Use Case Diagram Problem**

A Hospital Management System allows patients to book appointments, doctors to update medical records, and administrators to manage hospital resources. The requirements are not clearly documented.

### **a) Actors Identified**

The following actors interact with the Hospital Management System:

- Patient
- Doctor
- Administrator

### **(b) Primary Use Cases**

The primary use cases are:

- Login
- Book Appointment
- View Appointment
- Update Medical Record
- Manage Patients
- Manage Resources
- Logout

### **(c) Relationships**

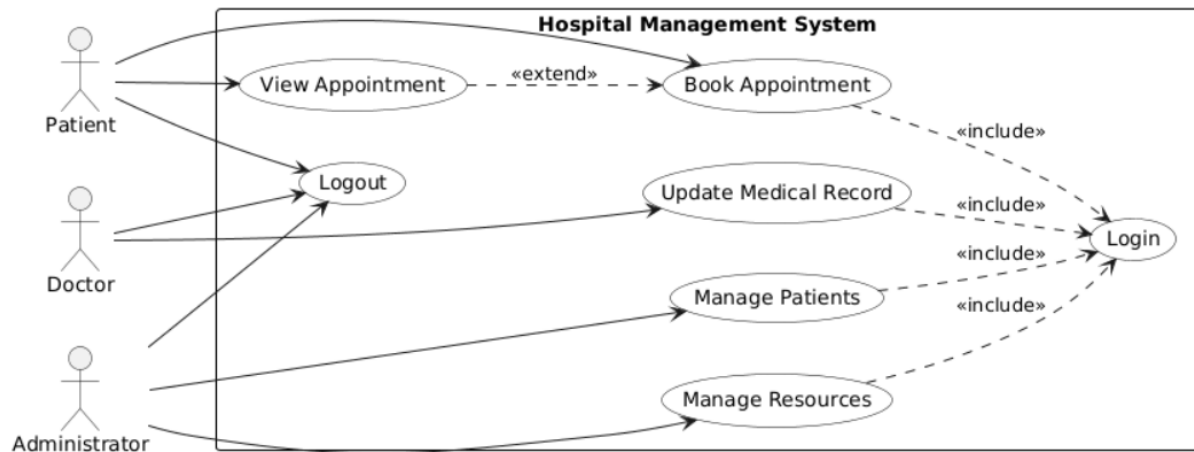
#### **Include Relationships**

- Book Appointment <<include>> Login
- Update Medical Record <<include>> Login
- Manage Patients <<include>> Login
- Manage Resources <<include>> Login

#### **Extend Relationship**

- View Appointment <<extend>> Book Appointment

#### (d) Use Case Diagram



#### Problem Understanding

The Hospital Management System is designed to simplify hospital operations by allowing patients to book appointments, doctors to update medical records, and administrators to manage patients and hospital resources. The Use Case Diagram identifies all users of the system and illustrates how they interact with different functionalities.

#### Method Selection

A Use Case Diagram is selected because it effectively captures the functional requirements of the Hospital Management System. It provides a clear visualization of the interactions between external actors and the system while representing the system boundary and relationships such as <<include>> and <<extend>>.

#### Solution Process

1. Identified the three primary actors: Patient, Doctor, and Administrator.
2. Determined the major system functionalities.
3. Connected each actor to the relevant use cases.
4. Applied <<include>> relationships to represent mandatory login before secure operations.
5. Applied an <<extend>> relationship to represent optional behavior for viewing appointments.
6. Enclosed all use cases within the Hospital Management System boundary using standard UML notation.

#### Interpretation

The Use Case Diagram provides a clear representation of the Hospital Management System by showing how each actor interacts with the available functionalities. It improves understanding of the system requirements, clearly separates user responsibilities, and supports effective communication between stakeholders during system design.

## **Conclusion**

The designed Use Case Diagram accurately models the functional requirements of the Hospital Management System. It follows UML standards by correctly identifying actors, use cases, system boundaries, and relationships, making it suitable for software analysis and design.

## 2. Class Diagram Problem

A Library Management System contains entities such as Book, Member, Librarian, and IssueRecord. The relationships among entities are not properly defined.

### (a) Classes with Attributes and Methods

#### Class 1: Book

##### Attributes

- BookID : Integer
- Title : String
- Author : String
- ISBN : String
- Status : String

##### Methods

- searchBook()
- updateStatus()

#### Class 2: Member

##### Attributes

- MemberID : Integer
- Name : String
- Phone : String
- Email : String

##### Methods

- borrowBook()
- returnBook()

#### Class 3: Librarian

##### Attributes

- StaffID : Integer
- Name : String
- Designation : String

##### Methods

- issueBook()
- collectBook()

## **Class 4: IssueRecord**

### **Attributes**

- IssueID : Integer
- IssueDate : Date
- ReturnDate : Date
- FineAmount : Double

### **Methods**

- createRecord()
- closeRecord()

### **(b) Relationships**

#### **Association**

- Member borrows Book.
- Librarian manages IssueRecord.

#### **Aggregation**

- IssueRecord contains Book information.

#### **Inheritance**

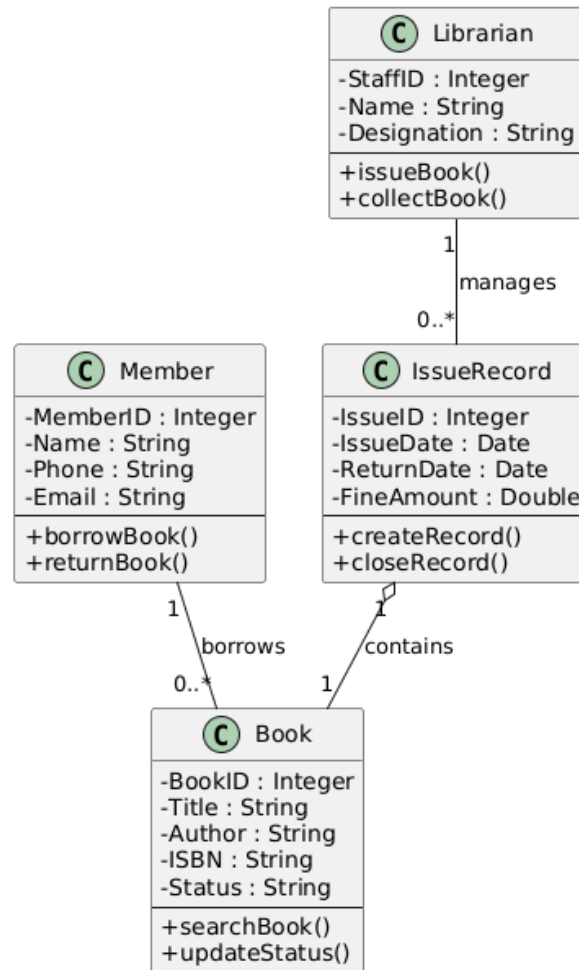
No inheritance relationship is required because all four classes represent independent entities with distinct responsibilities.

### **(c) Multiplicity**

| <b>Relationship</b>     | <b>Multiplicity</b> |
|-------------------------|---------------------|
| Member → Book           | 1 : 0..*            |
| Librarian → IssueRecord | 1 : 0..*            |
| IssueRecord → Book      | 1 : 1               |

#### (d) Class Diagram

**Library Management System - Class Diagram**



#### Problem Understanding

The Library Management System is designed to manage books, library members, librarians, and book issue records. The Class Diagram identifies the major classes, their attributes, methods, relationships, and multiplicities to represent the system's static structure.

#### Method Selection

A Class Diagram is selected because it represents the static structure of the system by showing classes, their attributes, operations, and relationships. It is the most suitable UML diagram for modeling object-oriented system architecture.

## **Solution Process**

1. Identified four primary classes: Book, Member, Librarian, and IssueRecord.
2. Assigned appropriate attributes and methods to each class.
3. Established associations between Member and Book, and between Librarian and IssueRecord.
4. Used **aggregation** between IssueRecord and Book to indicate that an issue record references a book.
5. Specified multiplicity to define how many objects participate in each relationship.
6. Designed the diagram using standard UML class notation.

## **Interpretation**

The Class Diagram clearly models the structural design of the Library Management System. It illustrates how objects interact, how data is organized, and how responsibilities are distributed among different classes. The use of multiplicity and aggregation improves clarity and accurately represents the relationships within the system.

## **Conclusion**

The designed Class Diagram effectively represents the Library Management System by defining its classes, attributes, methods, relationships, and multiplicities according to UML standards. It provides a clear blueprint for implementing the system in an object-oriented programming environment.

### **3. Activity Diagram Problem**

A Student Admission System includes application submission, document verification, fee payment, and enrollment confirmation. The workflow lacks proper decision-making steps.

#### **(a) Activities Identified**

The activities involved in the Student Admission System are:

- Fill Application Form
- Submit Application
- Upload Required Documents
- Verify Documents
- Pay Admission Fee
- Generate Student ID
- Assign Roll Number
- Confirm Enrollment

#### **(b) Decision Nodes**

The system contains one major decision node:

##### **Documents Verified?**

- **Yes** → Continue to Fee Payment.
- **No** → Reject Application and End Process.

#### **(c) Parallel Processing**

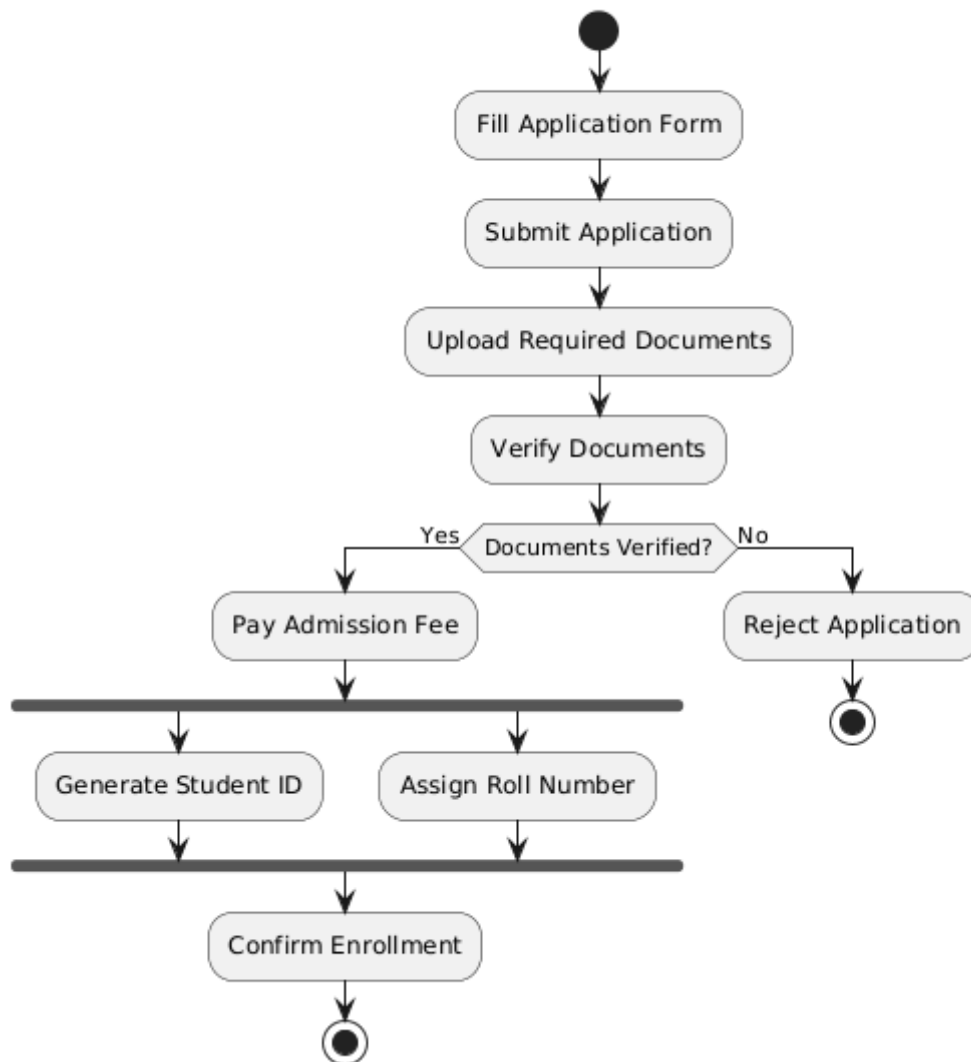
After successful fee payment, two activities occur simultaneously:

- Generate Student ID
- Assign Roll Number

Both activities are completed before enrollment confirmation.

#### (d) Activity Diagram

**Student Admission System - Activity Diagram**



#### Problem Understanding

The Student Admission System manages the admission process by accepting applications, verifying submitted documents, processing fee payments, and confirming student enrollment. The Activity Diagram illustrates the workflow, decision points, and parallel activities involved in the admission process.

#### Method Selection

An Activity Diagram is selected because it effectively models the workflow of the admission process. It clearly represents the sequence of activities, decision-making, parallel execution, and process completion using standard UML notation.

## **Solution Process**

1. Identified all major activities in the admission workflow.
2. Arranged the activities in their logical execution order.
3. Added a decision node to check whether the submitted documents are valid.
4. Included parallel processing after fee payment for generating the Student ID and assigning the Roll Number.
5. Joined the parallel activities before confirming enrollment.
6. Included both the initial and final nodes to complete the workflow.

## **Interpretation**

The Activity Diagram provides a clear representation of the Student Admission System workflow. It demonstrates how applications are processed, how decisions affect the flow, and how parallel tasks improve efficiency after fee payment. The diagram helps visualize the complete admission process and supports effective system design.

## **Conclusion**

The designed Activity Diagram accurately models the Student Admission System by representing activities, decision nodes, parallel processing, and workflow completion using standard UML notation. It provides a clear and optimized representation of the admission process suitable for software analysis and design.

#### 4. Component Diagram Problem

An Online Learning Platform consists of modules such as User Management, Course Management, Assessment Module, Content Delivery Module, and Notification Service.

##### (a) Major System Components

The major components of the Online Learning Platform are:

- User Management
- Course Management
- Assessment Module
- Content Delivery Module
- Notification Service

##### (b) Interfaces

The interfaces provided by each component are:

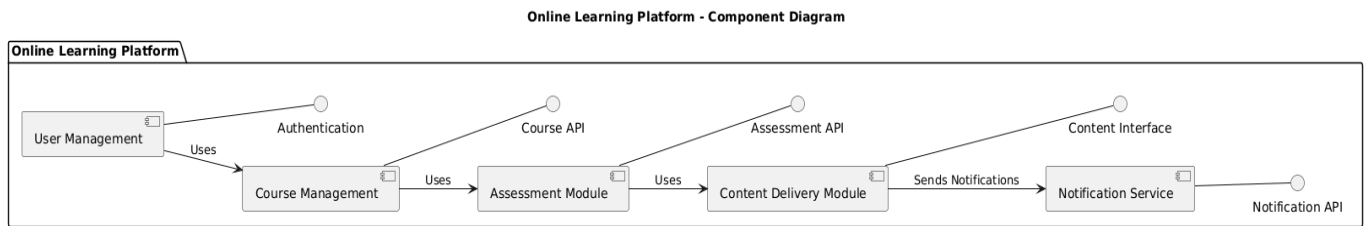
| Component               | Interface                |
|-------------------------|--------------------------|
| User Management         | Authentication Interface |
| Course Management       | Course API               |
| Assessment Module       | Assessment API           |
| Content Delivery Module | Content Interface        |
| Notification Service    | Notification API         |

##### (c) Dependencies

The dependencies among the components are:

- User Management depends on Course Management.
- Course Management depends on Assessment Module.
- Assessment Module depends on Content Delivery Module.
- Content Delivery Module depends on Notification Service.

## (d) Component Diagram



### Problem Understanding

The Online Learning Platform integrates multiple software components to provide user authentication, course management, online assessments, content delivery, and notification services. The Component Diagram illustrates the software architecture and the dependencies among these components.

### Method Selection

A Component Diagram is selected because it models the high-level architecture of the system by representing software modules, their interfaces, and dependencies. It helps developers understand how different components interact within the application.

### Solution Process

1. Identified the five major software components of the Online Learning Platform.
2. Assigned an interface to each component to represent the services it provides.
3. Established dependency relationships showing how components communicate and rely on one another.
4. Grouped all components within the system boundary using UML component notation.
5. Ensured the architecture follows a logical flow from user management to notification services.

### Interpretation

The Component Diagram provides a clear architectural view of the Online Learning Platform. It illustrates how individual software components collaborate through well-defined interfaces and dependencies. This modular design improves maintainability, scalability, and system integration while supporting efficient software development.

### Conclusion

The designed Component Diagram accurately represents the architecture of the Online Learning Platform by identifying components, interfaces, and dependencies using standard UML notation. It provides a clear blueprint for software implementation and system maintenance.